

00-playwright-notes



Playwright Notes



iFrame

Iframe is two separate html code file integrated into one using `iframe` element.

Handling iFrame

There are two ways to handle iframe.

1. frameLocator
2. frame object
3. contentFrame

```
<html>
  <iframe class="outerFrame" name="outerFrameName">
    <iframe class="innerFrame" name="innerFrameName">
      <button id="btn">Click me</button>
    </iframe>
  </iframe>
</html>
```

frameLocator

- auto waits for elements
- reduces flakiness
- works well with nested frames
- meta data is not present using frameLocator

```
const frame = await page.frameLocator(".innerFrame");
frame.locator("#btn").click();
```

frame Object

Using frame object we get extra methods to operate on frames such as

- frame name
- Url
- childFrame
- parentFrame

- isDetached
- page

Any frame can be selected using two methods in frame object

- by url
- by name

By URL

```
const frame = page.frame({ url: "https://frame-url.com" });
frame.locator("#btn").click();
```

By Name

```
const frame = page.frame("innerFrameName");
frame.locator("#btn").click();
```

Different Methods Present In Frame

```
const frame = page.frame("frameName");
frame.url();
frame.name();
frame.childFrame();
frame.parentFrame();
frame.isDetached();
```

All Frames

To get all frames we have method `page.frames()`. This will get all the frames inside main frame or we can also provide `name` or `url` inside `frames()` method.

```
const frames = page.frames();
```

Frame traversing

frame traversing doen by

- frame.paretnFrame()
- frame.page()

where `frame.pratentFrame()` it traverse immediate frame one level up and `frame.page()` it switch all context to main page instead of one step at a time.

frameLocator is most preferred way to interact with iframe in playwright. But they don't provide meta information like name, url, isDetached for that we use frame object.

iFrame in selenium

```
// 1. By index (order in the DOM, 0-based)
driver.switchTo().frame(0);

// 2. By name or id attribute
driver.switchTo().frame("frameNameOrId");

// 3. By WebElement (most robust - locate the iframe like any element)
WebElement frame = driver.findElement(By.cssSelector("iframe#myFrame"));
driver.switchTo().frame(frame);
```

The WebElement approach is preferred because index is fragile (breaks if frame order changes) and not every frame has a name/id.

- `defaultContent()` — jumps all the way back to the top-level page.
- `parentFrame()` — moves up just one level (useful for nested frames).

Waiting for a frame (recommended)

Frames may load asynchronously, so combine with `WebDriverWait`:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("myFrame")));
```

Common gotchas

- `NoSuchElementException` when an element "should" be there → you're usually in the wrong frame context. Check whether you forgot to `switchTo().frame(...)` or forgot to `switchTo().defaultContent()`.
- After switching into a frame, all subsequent `findElement` calls are scoped to that frame until you switch out.

Comparison Tables

Playwright: frameLocator vs frame Object

Aspect	frameLocator	frame Object
Auto-waits for elements	Yes	No
Flakiness	Low (reduces flakiness)	Higher

Aspect	frameLocator	frame Object
Nested frames	Works well	Supported but manual
Frame metadata (name, url, etc.)	Not available	Available
Selecting a frame	By selector	By name or url
Extra methods (childFrame, parentFrame, isDetached, page)	No	Yes
Best for	Simple element interaction	Inspecting/traversing frames

Selenium vs Playwright iFrame Handling

Aspect	Selenium (Java)	Playwright
Model	Stateful — switch context in/out	Stateless — chain straight in
Switch into frame	<code>driver.switchTo().frame(.)</code>	<code>page.frameLocator(.) /</code> <code>page.frame(.)</code>
Switch back out	Required (<code>defaultContent()</code> / <code>parentFrame()</code>)	Not needed
Auto-wait	No (need <code>WebDriverWait</code>)	Yes (built-in)
Select frame by	Index, name/id, WebElement	Selector, name, url
Scope after switching	All lookups scoped to frame until switch out	Only chained locator is scoped
Common pitfall	Wrong frame context / forgot to switch out	Few — no context to manage

Playwright Methods — Quick Revision

Method	What it does
<code>page.frameLocator(selector)</code>	Returns a frame locator (auto-waits) to act on elements inside the frame
<code>page.frame(name)</code>	Get a frame object by its name
<code>page.frame({ url })</code>	Get a frame object by its url
<code>page.frames()</code>	Returns array of all frames on the page
<code>frame.locator(selector)</code>	Locate an element inside the frame
<code>frame.url()</code>	Get the frame's url

Method	What it does
<code>frame.name()</code>	Get the frame's name
<code>frame.childFrame()</code>	Get the child (nested) frame
<code>frame.parentFrame()</code>	Move one level up to the parent frame
<code>frame.isDetached()</code>	Check if the frame is detached from DOM
<code>frame.page()</code>	Jump straight back to the main page context

Playwright → Selenium (Java) Method Mapping

Selenium has no first-class frame object, so metadata is read via `getAttribute(. )` or the JavaScript executor.

Playwright	Selenium (Java) equivalent
<code>frame.url()</code>	<code>frameElement.getAttribute("src")</code> or JS <code>document.URL</code>
<code>frame.name()</code>	<code>frameElement.getAttribute("name")</code> or JS <code>window.name</code>
<code>frame.parentFrame()</code>	<code>driver.switchTo().parentFrame()</code>
<code>frame.page()</code>	<code>driver.switchTo().defaultContent()</code>
<code>frame.childFrame()</code>	switch into the nested frame (<code>switchTo().frame(.)</code>)
<code>frame.isDetached()</code>	no direct equivalent (catch <code>StaleElementReferenceException</code>)
<code>page.frames()</code>	<code>driver.findElements(By.tagName("iframe"))</code> (returns elements, not frame objects)
<code>page.frameLocator(sel).locator(.)</code>	<code>switchTo().frame(.)</code> then <code>findElement(.)</code>

Dialog

There are three types of dialog in playwright,

- alert
- confirm
- prompt

By default playwright **auto dismisses** every dialog. To actually handle them we need to listen for the `dialog` event using an `dialog handler` .

```
page.on("dialog", async (d) => {  
  await d.accept();  
  await d.dismiss();  
  d.type();  
  d.message();  
  d.defaultValue();  
  d.page();  
});
```

*Important: the `dialog handler` is always written **before** the action that triggers the alert, so it does not miss the dialog.*

*Important: inside the handler you **must** call `d.accept()` or `d.dismiss()`. If you add a handler but never accept/dismiss, the dialog stays open and the test hangs.*

Events used to listen dialog

- `page.on()` ⇒ listens for the event continuously once activated
- `page.once()` ⇒ listens only once for the defined event
- `page.waitForEvent("dialog")` ⇒ waits for and returns the dialog (used when you want to listen inline)

Different methods in dialog

- `d.type()` ⇒ gives type of dialog (`alert` / `confirm` / `prompt`)
- `d.message()` ⇒ gives text present on the dialog
- `d.defaultValue()` ⇒ gives the default value of a `prompt` dialog
- `d.page()` ⇒ gives 
- `d.accept()` ⇒ clicks OK on the dialog
- `d.dismiss()` ⇒ dismisses (clicks Cancel) on the dialog
- `d.accept("john")` ⇒ sends text into a `prompt` input then accepts

simple alert

Simple alert with only an `OK` button. Used to show a warning message.

```
// starts listening for 'dialog' event
page.on("dialog", async (d) => {
  expect(d.type()).toContain("alert");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  // await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

confirm alert

Accept or reject type alert with `OK` and `Cancel` buttons.

```
// starts listening for 'dialog' event
page.on("dialog", async (d) => {
  expect(d.type()).toContain("confirm");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  // await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

prompt alert

Accept or reject type alert where the user can also send a `custom message` into an input.

```
// starts listening for 'dialog' event
page.on("dialog", async (d) => {
  expect(d.type()).toContain("prompt");
  expect(d.message()).toContain("alert message text");

  await d.accept("Kundalik"); // text passed into the prompt input
});

// click on alert button
await page.locator("#alert-btn").click();
```

Dialog Handler

Using the dialog handler we can interact with alerts. We can inspect the dialog `type` and `message`, then either `accept` or `dismiss` them, and for a `prompt` alert we can provide an input value.

page in dialog handler

```
// Listen globally across all tabs in this context
context.on("dialog", async (dialog) => {
  const sourcePage = dialog.page(); // Get the page that triggered it
  const currentUrl = sourcePage.url();

  if (currentUrl.includes("/checkout")) {
    console.log(`Critical alert on checkout page: ${dialog.message()}`);
    await dialog.accept();
  } else {
    await dialog.dismiss();
  }
});
```

Interview Questions

Q - How handle alert in playwright?

Q - Why do we add the dialog handler before clicking the alert button?

Ans:-

- The alert appears immediately when we click the alert button.
- JS is async in nature, so the alert can fire before a handler placed after the click is ready.
- To handle the alert we must already be listening on the `dialog` event; if the handler is placed after the click, we may lose the dialog.
- So the dialog handler is always placed before clicking the alert.

Q - What is the difference between `page.on()` and `page.once()`?

Ans:-

- `page.on()` ⇒ once activated, listens for **every** dialog
- `page.once()` ⇒ listens **only once** after it is activated

Q - What happens if we don't handle a dialog?

Ans:-

- Playwright auto dismisses it by default.
- But if a handler is registered and we never call `accept()` / `dismiss()`, the test will hang/timeout.



Alerts in Selenium

In Selenium Java an alert is handled using the `Alert` interface. We first switch to the alert, then act on it.


```
// switch to the alert
Alert alert = driver.switchTo().alert();

alert.accept();           // click OK
alert.dismiss();         // click Cancel
alert.getText();          // read the alert message
alert.sendKeys("kundalik"); // type into a prompt alert
```

Playwright vs Selenium — Alert Handling

Aspect	Playwright	Selenium (Java)
Model	Event driven (listen for <code>dialog</code>)	Switch to alert (<code>switchTo().alert()</code>)
When to set up	Handler before the trigger click	Switch after the alert appears
Default behaviour	Auto dismisses unless handled	Stays open until handled (else error)
Get type	<code>d.type()</code>	No direct method
Read message	<code>d.message()</code>	<code>alert.getText()</code>
Accept / OK	<code>d.accept()</code>	<code>alert.accept()</code>
Dismiss / Cancel	<code>d.dismiss()</code>	<code>alert.dismiss()</code>
Prompt input	<code>d.accept("text")</code>	<code>alert.sendKeys("text")</code> then <code>accept()</code>
Not handled	Auto dismissed	<code>UnhandledAlertException</code>

Playwright → Selenium (Java) Method Mapping

Playwright	Selenium (Java) equivalent
<code>page.on("dialog", handler)</code>	<code>driver.switchTo().alert()</code>
<code>d.message()</code>	<code>alert.getText()</code>
<code>d.accept()</code>	<code>alert.accept()</code>
<code>d.dismiss()</code>	<code>alert.dismiss()</code>
<code>d.accept("text")</code>	<code>alert.sendKeys("text")</code> then <code>alert.accept()</code>
<code>d.type()</code>	no direct equivalent
<code>d.defaultValue()</code>	no direct equivalent use <code>javaScript executor</code>

Reporting in Playwright

Playwright has different built in reports

- List ⇒ simple list in command line
- Dot ⇒ ..F
- html ⇒ html report is created
- line ⇒ 4 passed
- json
- junit
- github actions
- custom reports
 - allure
 - monocart
 - testsum
 - current JS
 - serenity JS

Report configuration

For config any report we need to add `report type` inside `playwright.config.js` file as shown below. Or we can run any report by passing argument in CLI as `npx playwright test--report=list`

```
// playwright.config.js
reporter: "list";

// for multiple report to generate
reporter: [["list"], ["dot"], ["html"]];

// reporter configuration settings
reporter: [
  // 1. Your customized HTML reporter
  [
    "html",
    {
      open: "never",
      outputFolder: "my-html-report",
      port: 3000,
    },
  ],
  // 2. Terminal-based live logging
  ["list"],
  // 3. JSON Reporter (Requires 'outputFile')
  ["json", { outputFile: "results/test-run.json" }],
  // 4. JUnit XML Reporter for CI/CD gates (Requires 'outputFile')
  ["junit", { outputFile: "results/junit.xml" }],
];
```

configuration options are

- Open: "always" | "never" | "on-failure"
- host: 8080
- outputFolder: "folder_name"
- outputFile: "results/test-run.json"

Allure Report

Allure report can be generated using allure npm package.

Steps to create allure report

- Install `pnpm add -D @playwright/test allure-playwright`
- add this `reporter:[['allure-playwright', {resultsDir:'my-allure-results'}]]` to `playwright.config.js`
- Install `CLI` command to run allure report as `pnpm install -g allure-commandline--save-dev`
- Run test and then run this command `allure generate my-allure-results -o allure-report--clean`
- To open allure report run this `allure open allure-report`

Selenium Reporting

Allure report used in selenium



Mouse, Keyboard Actions in Playwright

In Playwright, mouse and keyboard actions are available directly on the `locator` (high level) or on `page.mouse` / `page.keyboard` (low level). The common actions are:

- `hover`
- `hover and click`
- `right click`
- `double click`
- `drag and drop`
- `click and hold`
- `keyboard actions`

*Note: Unlike Selenium, Playwright does **not** have an `Actions` builder class. Each action is its own auto-waiting method on the locator, so you don't need a `perform()` call at the end.*

Mouse Hover

Used to hover over a dropdown, tooltip, or help text so it becomes visible.

```
await page.locator("#help").hover();
```

Right Click

Right click on any element using the `button` option.

```
await page.locator("#rightClickBtn").click({ button: "right" });
```

The `button` option accepts:

- `left` (default)
- `right`
- `middle`

Double Click

To perform a double click we use the `dblclick()` method.

```
await page.locator("#db-click").dblclick();
```

Drag and Drop

Playwright provides a built-in method `source.dragTo(target)`. There is also a low-level manual approach using `page.mouse`.

```
const source = page.locator("#src");
const target = page.locator("#trg");

// Approach 01 - manual (low level)
await source.hover();
await page.mouse.down(); // press mouse on source

await target.hover();
await page.mouse.up(); // release on target

// Approach 02 - built in (preferred)
await source.dragTo(target);
```

Keyboard actions

Playwright handles keys with `page.keyboard.press()`. Shortcuts are written with `+`.

```
// Shortcut / multiple keys together
await page.keyboard.press("Control+A");
await page.keyboard.press("Shift+A");

// Single key
await page.keyboard.press("a");
await page.keyboard.press("Enter");

// Type a whole string character by character (fires individual key events)
// `type()` is deprecated, use `pressSequentially()` instead
await page.locator("#search").pressSequentially("kundalik", { delay: 100 });
```

`press()` is one key/shortcut at a time. `pressSequentially()` types a full string character by character and fires real key events (the `delay` option, in ms, adds a pause between each key — useful for inputs that react on every keystroke). It replaces the deprecated `type()`. To fill a field quickly without per-key events, prefer `locator.fill("text")`.

Actions in Selenium Java

In Selenium all these actions are performed using the `Actions` class.

```
Actions act = new Actions(driver);
```

- The `Actions` class supports **method chaining**.
- It follows the **builder pattern**.

- We can chain multiple actions together, but to actually run them we must call `perform()` at the end.
- Chaining makes complex operations readable and efficient.

*Note: each block below is a **standalone example** of one gesture. If you ever run several gestures back-to-back on the **same** `act` instance, remember Selenium accumulates queued actions and `perform()` does not clear them — so use a fresh `Actions` instance per gesture in real tests. `Duration.ofSeconds(. )` needs `import java.time.Duration;`*

```
WebDriver driver = new ChromeDriver();
Actions act = new Actions(driver);

WebElement element = driver.findElement(By.id("help"));

// Mouse hover
act.moveToElement(element)
    .perform();

// Hover and click
act.moveToElement(element)
    .click()
    .perform();

// Right click
act.contextClick(element)
    .perform();

// Double click
act.doubleClick(element)
    .perform();

// Click and hold
act.clickAndHold(element)
    .perform();

// Click, hold, then release
act.clickAndHold(element)
    .pause(Duration.ofSeconds(2))
    .release()
    .perform();
```

Drag and Drop

To drag and drop we use the `dragAndDrop(source, target)` method with the source and target elements.

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.dragAndDrop(source, target)
    .perform();
```

Sometimes `dragAndDrop(src, trg)` won't work because of custom JS-based drag scripts that Selenium's method cannot capture. In that case use the manual click-hold-move-release approach:

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.clickAndHold(source)
    .moveToElement(target)
    .release()
    .perform();
```

Move using co-ordinates

To drag by a pixel offset instead of to a target element:

```
act.dragAndDropBy(source, xOffset, yOffset)
    .perform();
```

Here `xoffset` and `yoffset` are the distances to move in `px`.

KeyPress in Selenium

The `Keys` enum is used to perform keyboard operations in Selenium.

```
import org.openqa.selenium.Keys;

WebElement searchLoc = driver.findElement(By.name("q"));

act.sendKeys(searchLoc, "kundalik")
    .sendKeys(Keys.ENTER)
    .perform();
```

- `keyDown()` ⇒ simulates pressing and holding a key.
- `keyUp()` ⇒ simulates releasing the held key.

Performing Ctrl + A

```
act.keyDown(Keys.CONTROL) // press & hold Ctrl
    .sendKeys("a")         // press A
    .keyUp(Keys.CONTROL)  // release Ctrl
    .perform();
```

Interview Questions

Q - Why do we use `keyDown` before `keyUp`?

Ans:-

- `keyDown()` presses and holds the key.
- `keyUp()` releases the held key.
- This mimics how a real user interacts: they press and hold the first key, press the second key, then release. Without `keyUp`, the key stays logically held down.

Q - Why does Selenium need `perform()` but Playwright does not?

Ans:-

- Selenium's `Actions` uses the builder pattern — it only builds a queue of actions; `perform()` actually executes them.
- Playwright actions are individual auto-waiting methods that execute immediately when awaited, so no `perform()` is needed.

Q - Difference between `dragTo` / `dragAndDrop` and the manual approach?

Ans:-

- The built-in method is one line and works for standard HTML drag.
- The manual approach (`hover/down/move/up`) is used when the built-in method fails on custom JS drag-and-drop implementations.

Q - `hover()` use cases?

Ans:-

- Revealing hidden menus, dropdowns, tooltips, or help text that only appear on hover.

Action Methods Comparison

Action	Playwright	Selenium (Java)
Hover	<code>locator.hover()</code>	<code>act.moveToElement(el).perform()</code>
Hover + click	<code>locator.hover()</code> then <code>locator.click()</code>	<code>act.moveToElement(el).click().perform()</code>
Right click	<code>locator.click({ button: "right" })</code>	<code>act.contextClick(el).perform()</code>
Double click	<code>locator.dblclick()</code>	<code>act.doubleClick(el).perform()</code>
Click and hold	<code>page.mouse.down()</code>	<code>act.clickAndHold(el).perform()</code>
Release	<code>page.mouse.up()</code>	<code>act.release().perform()</code>

Action	Playwright	Selenium (Java)
Drag and drop	<code>source.dragTo(target)</code>	<code>act.dragAndDrop(src, trg).perform()</code>
Move by offset	<code>page.mouse.move(x, y)</code>	<code>act.dragAndDropBy(src, x, y).perform()</code>
Press key / shortcut	<code>page.keyboard.press("Control+A")</code>	<code>act.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform()</code>
Type string	<code>locator.pressSequentially("text", { delay: 100 })</code>	<code>act.sendKeys("text").perform()</code>
Needs perform()	No (auto executes)	Yes (builder pattern)

Keys Comparison

Selenium	Playwright
<code>Keys.ENTER</code>	<code>"Enter"</code>
<code>Keys.CONTROL</code>	<code>"Control"</code>
<code>Keys.SHIFT</code>	<code>"Shift"</code>
<code>Keys.TAB</code>	<code>"Tab"</code>
<code>Keys.ESCAPE</code>	<code>"Escape"</code>
<code>Keys.ARROW_DOWN</code>	<code>"ArrowDown"</code>

Checkbox and Radio

Radio Button

- Allows single selection.
- You can select only one option within a group.
- Clicking another option automatically deselects the previous one.
- Clicking an active radio button does nothing.
-  Radio button cannot be manually unchecked once checked. Playwright throws an error.

```
<input type="radio" />
```

Checkbox Button

- Allows multi-selection.

- You can select none, one, or multiple options within a group.
- Clicking an active checkbox unchecks it.

```
<input type="checkbox" />
```

Methods

- `check()` ⇒ to check any radio or checkbox
- `uncheck()` ⇒ to uncheck any radio or checkbox
- `setChecked(checked)` ⇒ to check or uncheck based on a boolean value
- `isChecked()` used to know whether it is checked or not
- `toBeChecked()` ⇒ to check whether it is checked or not
- `toBeEnabled()` ⇒ To ensure an element is interactive (not disabled)
- `toBeVisible()` ⇒ To verify an element is rendered on the page before trying to click it
- `toBeTruthy()` ⇒ check expected condition is true
- `toBeFalsy()` ⇒ check expected condition is false

```
// radio and checkbox check, uncheck, isChecked method
await page.locator("#loc").check();
await page.locator("#loc").uncheck();
await page.locator("#loc").isChecked();

// assertions on checkbox & radios
await expect(await page.locator("#loc").isChecked()).toBeTruthy();
await expect(await page.locator("#loc").isChecked()).toBeFalsy();

// preferred web-first assertion (auto-retries, no manual await needed)
await expect(page.locator("#loc")).toBeChecked();
await expect(page.locator("#loc")).not.toBeChecked();
```

setChecked()

`setChecked(checked)` is used when the desired state (checked/unchecked) is decided at runtime, e.g. coming from test data, a config flag, or a condition — instead of hard-coding a `check()` or `uncheck()` call. It sets the checkbox/radio to match the boolean passed in, and does nothing if it's already in that state.

```
// set state dynamically based on a boolean
const shouldBeChecked = true;
await page.locator("#loc").setChecked(shouldBeChecked);

// useful when driving state from test data
for (const item of [
  { loc: "#loc1", checked: true },
  { loc: "#loc2", checked: false },
]) {
  await page.locator(item.loc).setChecked(item.checked);
}
```

Multi checkBox

To check multiple records we first store them in an array and then perform check operation using a for...of loop.

```
// Check ops on multiple elements
const allCheckLoc = ["loc1"];

for (const checkLoc of allCheckLoc) {
  await page.locator(checkLoc).check();
}

// Uncheck on checked locators only
for (const checkLoc of allCheckLoc) {
  if (await page.locator(checkLoc).isChecked()) {
    await page.locator(checkLoc).uncheck();
  }
}
```



CheckBox & Radio in Selenium

In Selenium `click()` method is used to `check` and `uncheck` any radio or checkbox.

Checkbox and Radio Button Method Comparison

Action / Goal	Playwright (JS/TS)	Selenium (Java)
Check a Checkbox	<code>locator.check()</code>	<code>if(!el.isSelected()) el.click();</code>
Uncheck a Checkbox	<code>locator.uncheck()</code>	<code>if(el.isSelected()) el.click();</code>
Select a Radio Button	<code>locator.check()</code>	<code>if(!el.isSelected()) el.click();</code>
Get Selection State	<code>await locator.isChecked()</code>	<code>element.isSelected()</code>

Action / Goal	Playwright (JS/TS)	Selenium (Java)
Assert State is Checked	<code>await expect(locator).toBeChecked()</code>	<code>assertTrue(element.isSelected());</code>
Assert State is Unchecked	<code>await expect(locator).not.toBeChecked()</code>	<code>assertFalse(element.isSelected());</code>
Verify if Clickable / Enabled	<code>await locator.isEnabled()</code>	<code>element.isEnabled()</code>
Force Check (Bypass UI overlays)	<code>locator.check({ force: true })</code>	<code>js.executeScript("arguments[0].click();", el);</code>

Key Behavioral Differences

Behaviour	Playwright	Selenium
Check / Uncheck	Idempotent — won't click if already in the target state.	<code>click()</code> always toggles state; needs a manual <code>isSelected()</code> guard.
Radio vs Checkbox	Same <code>.check()</code> API works for both radios and checkboxes.	<code>.click()</code> used for everything, no distinction.
Selection State	<code>isChecked()</code> returns a <code>Promise<boolean></code> (must <code>await</code>).	<code>isSelected()</code> returns a plain <code>boolean</code> synchronously.
Assertions	<code>toBeChecked()</code> is web-first and auto-retries until true or timeout.	<code>assertTrue</code> / <code>assertFalse</code> check the state once, immediately.
Enabled Check	Auto-checks actionability (enabled, visible, stable) before every action.	Requires an explicit <code>isEnabled()</code> call; no auto-waiting.
Force Actions	<code>{ force: true }</code> option bypasses actionability checks.	No equivalent option; falls back to <code>JavascriptExecutor</code> .

Playwright Concepts

Synchronous and Asynchronous in Playwright Locators

Creating a Playwright locator is just like creating an object or variable in JavaScript. It stays inside the Node.js memory and does nothing with the browser, so creating it is **synchronous** — no `await` needed.

A locator is just a variable that holds a **description of how to find a web element**, stored in memory.

```
const username = page.locator("#username");
```

But when we perform an action on the locator it becomes **asynchronous** because:

- Playwright starts interacting with the browser using its APIs.
- Those APIs take time, so the call becomes async, and to wait for the promise to resolve we use `await`.

```
await username.click();
```

Storing the locator in memory

```
const username = page.locator("#username");
```

- When this line runs, Playwright does nothing with the browser.
- It just stores the locator by creating a variable in node js memory.
- No network call to the browser.
- No DOM query happens.
- No searching of the page.
- It only creates a JavaScript object in memory.

Performing action locator

```
await username.click();`
```

- When the user performs an action on the locator, the real work starts.
- Playwright starts interacting with the browser using its APIs over a WebSocket connection.
- Playwright sends a message over the WebSocket, then
- The browser starts searching the DOM for the locator.
- For searching the locator in DOM we required to wait for the network call to complete.
- So we use `await` to wait until the element is:
 - attached
 - visible
 - stable
 - enabled
 - editable
- All this communication happens via IPC (Inter-Process Communication) over CDP (Chrome DevTools Protocol), which takes time and happens over the network — asynchronous in nature.

*Key point: Playwright resolves the locator **at the time of the action**, not when it is created. This is why Playwright does not throw `StaleElementReferenceException` like Selenium — the element is freshly queried every time you act on it.*

Parameter	Selenium	Playwright
findElement/locator	immediately queries DOM	stores as a reference in memory
returns	actual element reference	a lazy locator object
sync or async	sync	sync (stays in Node process)
stale element click	Yes ⇒ uses old reference	No ⇒ queries at time of action